

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
SCC-541 – Laboratório de Bases de Dados

Relatório – Trabalho Prático T4

Visões em PostgreSQL

Katiely Feitosa de Lacerda – 12777100
Leonardo Gonsalez – 15657074
Miguel Filippo Calhabeu – 15480331
Renan Silva Soriano – 11794824
Guilherme Motta Tranche – 13671549

São Carlos – 2026

0 Introdução

Este relatório apresenta o desenvolvimento e os testes realizados para o Trabalho Prático T4 da disciplina SCC-541 – Laboratório de Bases de Dados, sobre visões (views) em PostgreSQL. A base utilizada é a mesma construída ao longo da disciplina, composta por dados da Fórmula 1, cidades e aeroportos do mundo.

O objetivo principal foi explorar os diferentes tipos de visões oferecidos pelo PostgreSQL:

- Visões materializadas (MATERIALIZED VIEW) – armazenam fisicamente o resultado da consulta, com análise de desempenho e uso de REFRESH.
- Visões transientes (VIEW) – recalculadas a cada acesso, utilizadas para filtrar e combinar dados dinamicamente.
- Visões inline (WITH / CTE) – empregadas para isolar cálculos de distância e evitar repetição de código.

Foram desenvolvidas cinco visões principais:

- Aeroportos_Brasileiros – visão materializada com aeroportos de cidades brasileiras;
- Aeroportos_sem_cidades – aeroportos sem city_id associado;
- Cidades_brasileiras – cidades do Brasil com população $\geq 100\,000$ hab.;
- Problemas_aeroportos – aeroportos sem cidade a ≤ 10 km de cidades brasileiras;
- Correcao_aeroportos – visão atualizável usada para corrigir city_id.

Também foi criada a visão Circuitos_completa (Exercício 3), que consolida informações geográficas completas dos circuitos de Fórmula 1.

1 Exercício 1 – Visão Materializada Aeroportos_Brasileiros

1.1 Resolução

A visão materializada Aeroportos_Brasileiros foi criada a partir das tabelas airports, cities e countries, filtrando pelo código de país BR. Os atributos selecionados são nome e coordenadas do aeroporto, nome e continente do país e nome e população da cidade vinculada.

A escolha por visão materializada é justificada pelo volume dos dados: a tabela airports contém mais de 84 000 registros, e as junções com cities e countries são custosas. Armazenar o resultado pré-computado reduz significativamente o tempo de consulta.

```
CREATE MATERIALIZED VIEW Aeroportos_Brasileiros AS
    SELECT a.name, a.latitude_deg, a.longitude_deg,
    co.name AS pais_nome, cont.name AS continente, ci.name AS cidade_nome, ci.population
    AS cidade_populacao
    FROM airports a
    JOIN cities ci ON ci.id = a.city_id
    JOIN countries co ON co.id = ci.country_id
    WHERE co.code = 'BR';
```

1.2 Testes e Resultados

1.2.1 Exemplo de tuplas e contagem

A consulta `SELECT * FROM Aeroportos_Brasileiros LIMIT 10` retorna os primeiros aeroportos brasileiros cadastrados. A contagem total é obtida com `SELECT COUNT(*)`.

A visão contém 7 828 aeroportos brasileiros, incluindo pequenos aeródromos e heliportos.

```

DROP MATERIALIZED VIEW IF EXISTS Aeroportos_Brasileiros;

CREATE MATERIALIZED VIEW Aeroportos_Brasileiros AS
SELECT
    a.name           AS aeroporto_nome,
    a.latitude_deg,
    a.longitude_deg,
    co.name          AS pais_nome,
    cont.name        AS continente,
    ci.name          AS cidade_nome,
    ci.population    AS cidade_populacao
FROM airports a
JOIN cities ci ON ci.id = a.city_id
JOIN countries co ON co.id = ci.country_id
JOIN continents cont ON cont.id = co.continent_id
WHERE co.code = 'BR';

-- Exemplo: primeiras 10 tuplas
SELECT * FROM Aeroportos_Brasileiros LIMIT 10;

-- Número total de tuplas
SELECT COUNT(*) AS total_aeroportos_brasileiros FROM Aeroportos_Brasileiros;

```

Output: materialized view "aeroportos_brasil"

	AZ aeroporto_nome	123 latitude_deg	123 longitude_deg	AZ pais_nome	AZ continente	AZ cidade_nome	123 cidade_populacao
1	Escarpas Heliport	-16.212867	-48.509056	Brazil	South America	Abadiânia	[NULL]
2	Fazenda Mandarim Airport	-1.794319	-48.792022	Brazil	South America	Abaetetuba	158,188
3	JMG Agropecuária Airstrip	-19.023319	-45.4117	Brazil	South America	Abaeté	22,675
4	Abaeté Airport	-19.1556	-45.494801	Brazil	South America	Abaeté	22,675
5	Jeová Gomes Corrêa Airport	-8.73699	-39.115398	Brazil	South America	Abaré	17,639
6	Fazenda Vale do Sol II Airstrip	-9.246739	-49.395683	Brazil	South America	Abreulândia	[NULL]
7	Fazenda São Luiz Airport	-9.386899	-49.310819	Brazil	South America	Abreulândia	[NULL]
8	Fazenda 4 Amigos Airport	-9.244835	-49.666942	Brazil	South America	Abreulândia	[NULL]
9	Abunã Airstrip	-9.71045	-65.35793	Brazil	South America	Abunã	[NULL]
10	Fazenda Santa Cármen Airstrip	-9.719693	-65.144742	Brazil	South America	Abunã	[NULL]

1.2.2 Comparação de desempenho

Foram executadas pelo menos 5 consultas com EXPLAIN ANALYZE tanto na visão materializada quanto na consulta equivalente nas tabelas-base. Os resultados médios observados foram:

- Consulta direta nas tabelas-base: ~29,6 ms (3 Hash Joins + Seq Scans, EXPLAIN ANALYZE).
- Consulta na visão materializada: ~0,8 ms (Seq Scan direto na tabela materializada).
- Ganho de desempenho: fator de aproximadamente 37x (29,6 ms / 0,8 ms).
- Consumo de armazenamento da visão materializada: 936 kB (pg_total_relation_size).

A principal desvantagem é que a visão materializada não reflete alterações nas tabelas-base automaticamente — é necessário executar REFRESH MATERIALIZED VIEW.


```
-- Consulta pela visão materializada (deve ser mais rápida)
EXPLAIN ANALYZE
SELECT * FROM Aeroportos_Brasileiros;

-- Consumo de armazenamento da visão materializada
SELECT
    pg_size_pretty(pg_total_relation_size('aeroportos_brasileiros')) AS tamanho_visao_mat;
```

results 1 ×

EXPLAIN ANALYZE SELECT * FROM Aeroportos_Brasileiros | Enter a SQL expression to filter results (use Ctrl+Space)

	AZ QUERY PLAN
1	Seq Scan on aeroportos_brasileiros (cost=0.00..190.28 rows=7828 width=82) (actual time=0.009..0.462 rows=7828 loops=1)
2	Planning Time: 0.032 ms
3	Execution Time: 0.766 ms

```
-- Consumo de armazenamento da visão materializada
SELECT
    pg_size_pretty(pg_total_relation_size('aeroportos_brasileiros')) AS tamanho_visao_mat;
```

Results 1 ×

SELECT pg_size_pretty(pg_total_relation_size('aeroportos_brasileiros')) | Enter a SQL expression to filter results (use Ctrl+Space)

	AZ tamanho_visao_mat
1	936 kB

1.2.3 Demonstração do REFRESH

Para demonstrar a necessidade do REFRESH, foi inserido um aeroporto fictício vinculado a uma cidade brasileira. A consulta à visão antes do REFRESH retorna 0 registros com o nome fictício; após o REFRESH, o registro aparece corretamente.

```
-- Antes do REFRESH: visão desatualizada
SELECT COUNT(*) FROM Aeroportos_Brasileiros WHERE aeroporto_nome LIKE '%T4%';
-- Resultado: 0
```

```
REFRESH MATERIALIZED VIEW Aeroportos_Brasileiros;
```

```
-- Após o REFRESH: visão atualizada
SELECT COUNT(*) FROM Aeroportos_Brasileiros WHERE aeroporto_nome LIKE '%T4%';
-- Resultado: 1
```

The screenshot displays two instances of a database query results window, labeled 'Results 1'. The top window shows the query `SELECT COUNT(*) AS antes_do_refresh FROM Aeroportos_Brasileiros` with a result of 0. The bottom window shows the query `SELECT COUNT(*) AS apos_o_refresh FROM Aeroportos_Brasileiros` with a result of 1. Both windows have a 'Grid' view selected and a dropdown menu set to '123 antes_do_refresh' and '123 apos_o_refresh' respectively.

antes_do_refresh
0

apos_o_refresh
1

2 Exercício 2 – Aeroportos sem Cidade e Associação por Distância

2.1 Resolução

Foram criadas três visões para este exercício:

2.1.1 Aeroportos_sem_cidades

Filtra aeroportos com city_id IS NULL. Por definição, esses aeroportos não podem ser classificados por país a partir da modelagem atual.

```
CREATE VIEW Aeroportos_sem_cidades AS
  SELECT id, name, latitude_deg, longitude_deg
  FROM airports
  WHERE city_id IS NULL;
```

2.1.2 Cidades_brasileiras

Seleciona cidades do Brasil com população de no mínimo 100 000 habitantes, incluindo nome, população e coordenadas.

```
CREATE VIEW Cidades_brasileiras AS
  SELECT ci.id, ci.name, ci.population, ci.latitude, ci.longitude
  FROM cities ci
  JOIN countries co ON co.id = ci.country_id
  WHERE co.code = 'BR' AND ci.population >= 100000;
```

2.1.3 Associação por distância ≤ 10 km

A função Earth_Distance (extensão EarthDistance + CUBE) calcula a distância em metros entre dois pontos. Usamos uma visão inline (WITH) para isolar o cálculo e evitar repetição:

```
WITH distancias AS (
  SELECT a.name AS aeroporto_nome, c.name AS cidade_nome,
         c.population,
         earth_distance(
           ll_to_earth(a.latitude_deg, a.longitude_deg),
           ll_to_earth(c.latitude, c.longitude)
         ) / 1000.0 AS distancia_km
  FROM Aeroportos_sem_cidades a
  CROSS JOIN Cidades_brasileiras c
)
SELECT * FROM distancias WHERE distancia_km <= 10
ORDER BY aeroporto_nome, distancia_km;
```

2.2 Testes

A consulta retorna os pares (aeroporto sem cidade, cidade brasileira) cuja distância é inferior ou igual a 10 km, apresentando nome do aeroporto, nome da cidade, população e distância arredondada em km.

A consulta retornou 1 aeroporto: "Guarulhos - Sao Paulo Airport Marriott Heliport" (id 275301), com 10 cidades brasileiras candidatas a ≤ 10 km. A mais próxima é Guarulhos (1

169 577 hab., 4,313 km).

-- Associação: aeroportos sem cidade x cidades brasileiras a ≤ 10 km
 -- Usa CTE para isolar o cálculo da distância

```
WITH distancias AS (
  SELECT
    a.name                                AS aeroporto_nome,
    c.name                                AS cidade_nome,
    c.population                          AS populacao,
    earth_distance(
      ll_to_earth(a.latitude_deg, a.longitude_deg),
      ll_to_earth(c.latitude,    c.longitude)
    ) / 1000.0                            AS distancia_km
  FROM Aeroportos_sem_cidades a
  CROSS JOIN Cidades_brasileiras c
)
SELECT aeroporto_nome, cidade_nome, populacao,
       ROUND(distancia_km::numeric, 3) AS distancia_km
FROM distancias
WHERE distancia_km <= 10
ORDER BY aeroporto_nome, distancia_km;
```

aeroportos_sem_cidades(+) 1 X

WITH distancias AS (SELECT a.name AS aeroporto_nome, c.name AS c | Enter a SQL expression to filter results (use Ctrl+Space)

	AZ aeroporto_nome	AZ cidade_nome	123 populacao	123 distancia_km
1	Guarulhos - Sao Paulo Airport Marriott Heliport	Guarulhos	1,169,577	4.313
2	Guarulhos - Sao Paulo Airport Marriott Heliport	Ermelino Matarazzo	112,333	4.71
3	Guarulhos - Sao Paulo Airport Marriott Heliport	Cangaiba	141,172	5.443
4	Guarulhos - Sao Paulo Airport Marriott Heliport	Vila Jacui	134,189	6.065
5	Guarulhos - Sao Paulo Airport Marriott Heliport	Jardim Helena	129,409	8.619
6	Guarulhos - Sao Paulo Airport Marriott Heliport	Artur Alvim	104,864	9.418
7	Guarulhos - Sao Paulo Airport Marriott Heliport	Vila Medeiros	114,839	9.502
8	Guarulhos - Sao Paulo Airport Marriott Heliport	Vila Matilde	103,558	9.683
9	Guarulhos - Sao Paulo Airport Marriott Heliport	Itaquera	210,960	9.779
10	Guarulhos - Sao Paulo Airport Marriott Heliport	Vila Curuca	140,673	9.846

3 Exercício 3 – Visão Circuitos_completa

3.1 Resolução

A visão Circuitos_completa consolida as informações de localização geográfica de todos os circuitos de Fórmula 1 cadastrados, independentemente de estarem vinculados a uma cidade. Para isso, foram utilizados LEFT JOINS, garantindo que circuitos sem city_id também apareçam no resultado.

```
CREATE VIEW Circuitos_completa AS
SELECT ci.name AS circuito_nome, ci.lat, ci.long,
       ct.name AS cidade_nome,
       co.name AS pais_nome, co.code AS pais_code,
       cont.name AS continente_nome
FROM circuits ci
LEFT JOIN cities      ct ON ct.id = ci.city_id
LEFT JOIN countries  co ON co.id = ct.country_id
LEFT JOIN continents cont ON cont.id = co.continent_id;
```

A escolha por LEFT JOIN em vez de INNER JOIN é justificada pelo enunciado: "deve apresentar todos os circuitos, independentemente de estarem ou não vinculados a alguma cidade".

3.2 Testes

Número de tuplas: a visão contém 76 tuplas (um circuito por linha), correspondendo a todos os circuitos cadastrados na tabela circuits.

A consulta SELECT * FROM Circuitos_completa LIMIT 10 retorna exemplos variados.

The screenshot shows a SQL IDE with the following SQL code in the editor:

```
DROP VIEW IF EXISTS Circuitos_completa;
CREATE VIEW Circuitos_completa AS
SELECT
    ci.name AS circuito_nome,
    ci.lat,
    ci.long,
    ct.name AS cidade_nome,
    co.name AS pais_nome,
    co.code AS pais_code,
    cont.name AS continente_nome
FROM circuits ci
-- LEFT JOIN para incluir circuitos sem cidade vinculada
LEFT JOIN cities ct ON ct.id = ci.city_id
LEFT JOIN countries co ON co.id = ct.country_id
LEFT JOIN continents cont ON cont.id = co.continent_id;

-- Número de tuplas
SELECT COUNT(*) AS total_circuitos FROM Circuitos_completa;

-- Exemplo variado: primeiras 10 tuplas
SELECT * FROM Circuitos_completa LIMIT 10;
```

The output window shows the results of the last query (SELECT * FROM Circuitos_completa LIMIT 10). The results are displayed in a table with the following columns: circuito_nome, lat, long, cidade_nome, pais_nome, pais_code, and continente_nome.

	AZ circuito_nome	lat	long	AZ cidade_nome	AZ pais_nome	AZ pais_code	AZ continente_nome
1	Ain Diab	33.5786	-7.6875	Casablanca	Chile	CL	South America
2	Adelaide Street Circuit	-34.9272	138.617	Adelaide	South Africa	ZA	Africa
3	Circuito de Madrid Jarama	40.6171	-3.58558	Madrid	United States	US	North America
4	Valencia Street Circuit	39.4589	-0.331667	Valencia	Spain	ES	Europe
5	Monsanto Park Circuit	38.7197	-9.20306	Lisbon	Portugal	PT	Europe
6	Pescara Circuit	42.475	14.1508	Pescara	Italy	IT	Europe
7	Riverside International Raceway	33.937	-117.273	California	United States	US	North America
8	Autódromo Oscar y Juan Gálvez	-34.6943	-58.4593	Buenos Aires	Argentina	AR	South America
9	Long Beach	33.7651	-118.189	California	United States	US	North America
10	Montjuïc circuit	41.3664	2.15167	Barcelona	Venezuela	VE	South America

4 Exercício 4 – Visão Problemas_aeroportos

4.1 Resolução

A visão Problemas_aeroportos filtra, a partir de Aeroportos_sem_cidades, apenas aqueles que possuem ao menos uma cidade da visão Cidades_brasileiras a ≤ 10 km de distância. O cálculo de distância é encapsulado em uma CTE para maior clareza:

```
CREATE VIEW Problemas_aeroportos AS
WITH distancias AS (
    SELECT a.id, a.name, a.latitude_deg, a.longitude_deg,
           c.name AS cidade_candidata, c.population,
           earth_distance(
               ll_to_earth(a.latitude_deg, a.longitude_deg),
               ll_to_earth(c.latitude, c.longitude)
           ) / 1000.0 AS distancia_km
    FROM Aeroportos_sem_cidades a
    CROSS JOIN Cidades_brasileiras c
)
SELECT id, name, latitude_deg, longitude_deg,
       cidade_candidata, population AS populacao,
       ROUND(distancia_km::numeric, 3) AS distancia_km
FROM distancias
WHERE distancia_km <= 10;
```

4.2 Testes e Explicação

(b) Por que esses aeroportos aparecem nessa visão? Os aeroportos listados possuem city_id NULL na tabela airports — ou seja, a base não registra uma cidade vinculada a eles. Entretanto, suas coordenadas geográficas (latitude e longitude) estão a menos de 10 km de cidades brasileiras com população de ao menos 100 000 habitantes. Esse cenário indica prováveis inconsistências de carga de dados: o aeroporto existe fisicamente próximo a uma grande cidade, mas a associação não foi inserida corretamente na base.

Para cada tupla retornada, o campo cidade_candidata e a distância em km indicam qual cidade brasileira é a mais provável candidata a ser vinculada ao aeroporto.

```

DROP VIEW IF EXISTS Problemas_aeroportos;
CREATE VIEW Problemas_aeroportos AS
WITH distancias AS (
    SELECT
        a.id AS aeroporto_id,
        a.name AS aeroporto_nome,
        a.latitude_deg,
        a.longitude_deg,
        c.name AS cidade_candidata,
        c.population AS populacao_candidata,
        earth distance(
            ll to earth(a.latitude_deg, a.longitude_deg),
            ll to earth(c.latitude, c.longitude)
        ) / 1000.0 AS distancia_km
    FROM Aeroportos_sem_cidades a
    CROSS JOIN Cidades_brasileiras c
)
SELECT
    aeroporto_id AS id,
    aeroporto_nome AS name,
    latitude_deg,
    longitude_deg,
    cidade_candidata,
    populacao_candidata AS populacao,
    ROUND(distancia_km::numeric, 3) AS distancia_km
FROM distancias
WHERE distancia_km <= 10;

-- Exemplo: primeiras tuplas da visão
SELECT * FROM Problemas_aeroportos ORDER BY name, distancia_km LIMIT 10;
        
```

Output

materialized view "aeroportos_brasileiros" does not exist, skip

view "aeroportos_sem_cidades" does not exist, skip

view "cidades_brasileiras" does not exist, skip

view "circuitos_completa" does not exist, skip

problemas_aeroportos 1

SELECT * FROM Problemas_aeroportos ORDER BY name, distancia_km

	id	name	latitude_deg	longitude_deg	cidade_candidata	populacao	distancia_km
1	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Guarulhos	1,169,577	4.313
2	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Ermelino Matarazzo	112,333	4.71
3	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Cangaiba	141,172	5.443
4	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Vila Jacui	134,189	6.065
5	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Jardim Helena	129,409	8.619
6	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Artur Alvim	104,864	9.418
7	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Vila Medeiros	114,839	9.502
8	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Vila Matilde	103,558	9.683
9	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Itaquera	210,960	9.779
10	275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	-23.4557270361	-46.4917990565	Vila Curuca	140,673	9.846

5 Exercício 5 – Visão Correcao_aeroportos e Atualização

5.1 Resolução

A visão Correcao_aeroportos seleciona diretamente de airports os atributos id, name, latitude_deg, longitude_deg e city_id, sem nenhuma junção, filtrada pelos aeroportos identificados em Problemas_aeroportos. Por derivar de uma única tabela sem agregações ou operações de conjunto, ela é automaticamente atualizável (AUVIS) no PostgreSQL.

```
CREATE VIEW Correcao_aeroportos AS
  SELECT id, name, latitude_deg, longitude_deg, city_id
  FROM airports
  WHERE id IN (SELECT DISTINCT id FROM Problemas_aeroportos);
```

5.2 Identificação de Cidades Candidatas e UPDATE

Para cada aeroporto presente em Correcao_aeroportos, foi selecionada a cidade mais próxima (menor distância) dentre as cidades da visão Cidades_brasileiras. O UPDATE é realizado diretamente na visão:

```
WITH cidade_mais_proxima AS (
  SELECT DISTINCT ON (id)
    id AS aeroporto_id,
    (SELECT c.id FROM cities c
     JOIN countries co ON co.id = c.country_id
     WHERE co.code = 'BR' AND c.name = p.cidade_candidata
     LIMIT 1) AS cidade_id
  FROM Problemas_aeroportos p
  ORDER BY id, distancia_km
)
UPDATE Correcao_aeroportos ca
SET city_id = cmp.cidade_id
FROM cidade_mais_proxima cmp
WHERE ca.id = cmp.aeroporto_id
AND cmp.cidade_id IS NOT NULL;
```

5.3 Verificação dos Efeitos

Após o UPDATE, as visões Aeroportos_sem_cidades e Problemas_aeroportos são consultadas novamente para verificar que os aeroportos corrigidos não aparecem mais:

```
-- Deve retornar 0 para os aeroportos corrigidos
SELECT COUNT(*) FROM Aeroportos_sem_cidades
WHERE id IN (SELECT id FROM Correcao_aeroportos);
```

```
-- Verificar que os aeroportos corrigidos não aparecem mais em Aeroportos_sem_cidades
SELECT COUNT(*) AS aeroportos_sem_cidade_restantes FROM Aeroportos_sem_cidades
WHERE id IN (SELECT id FROM Correcao_aeroportos);

-- Consulta final para confirmar as correções realizadas
SELECT ca.id, ca.name, ca.city_id, ci.name AS cidade_vinculada
FROM Correcao_aeroportos ca
JOIN cities ci ON ci.id = ca.city_id
ORDER BY ca.name
LIMIT 20;
```

Results 1 X

SELECT COUNT(*) AS aeroportos_sem_cidade_restantes FROM Aeroportos_sem_cidades

	123 aeroportos_sem_cidade_restantes
1	0

```
-- Confirma city_id atualizado
SELECT ca.id, ca.name, ca.city_id, ci.name AS cidade_vinculada
FROM Correcao_aeroportos ca
JOIN cities ci ON ci.id = ca.city_id
ORDER BY ca.name LIMIT 20;
```

Resultado real: 1 aeroporto corrigido — id 275301 ("Guarulhos - Sao Paulo Airport Marriott Heliport") recebeu city_id = 3461786 (Guarulhos). Após o UPDATE, Problemas_aeroportos retorna 0 linhas.

```
-- Consulta final para confirmar as correções realizadas
-- (usa airports diretamente; Correcao_aeroportos está vazia após a correção)
SELECT a.id, a.name, a.city_id, ci.name AS cidade_vinculada
FROM airports a
JOIN cities ci ON ci.id = a.city_id
WHERE a.id IN (SELECT id FROM ids_corrigidos)
ORDER BY a.name
LIMIT 20;
```

airports(+) 1 airports(+) 2 X

SELECT a.id, a.name, a.city_id, ci.name AS cidade_vinculada FROM airports a JOIN cities ci ON ci.id = a.city_id WHERE a.id IN (SELECT id FROM ids_corrigidos) ORDER BY a.name LIMIT 20;

123 id	A-Z name	123 city_id	A-Z cidade_vinculada
275,301	Guarulhos - Sao Paulo Airport Marriott Heliport	3,461,786	Guarulhos

6 Conclusão

Este trabalho demonstrou na prática as diferenças e aplicabilidades dos tipos de visões disponíveis no PostgreSQL:

- reduziu o tempo de consulta em ~37x (de ~29,6 ms para ~0,8 ms) ao pré-computar as junções, mas exige REFRESH manual para refletir alterações nas tabelas-base.
- Visões transientes (Aeroportos_sem_cidades, Cidades_brasileiras, Circuitos_completa, Problemas_aeroportos): sempre refletem o estado atual das tabelas-base, mas têm custo de execução a cada acesso.
- Visão atualizável (Correcao_aeroportos): permitiu corrigir o atributo city_id diretamente via UPDATE na visão, aproveitando a propriedade AUVIS do PostgreSQL para visões derivadas de uma única tabela.
- Visões inline (WITH/CTE): utilizadas para isolar o cálculo de distância Haversine via Earth_Distance, melhorando a legibilidade e evitando repetição de código.

As decisões adotadas buscaram manter as soluções aderentes ao esquema real da base, evitar alterações desnecessárias na estrutura e centralizar a lógica nos próprios objetos do SGBD.